Challenge Computer Vision – CIFAR-10

Matthieu GAUDRY

30 octobre 2026

Table des matières

1 Introduction du projet, jeu de données et objectifs **3**
1.1 Contexte . . . 3
1.2 Le jeu de données CIFAR-10 . . . 3
1.3 Découpage des données . . . 3
1.4 Objectifs . . . 3

2 Structure générale du code **4**
2.1 Vue d'ensemble . . . 4
2.2 La configuration : `TrainConfig` . . . 4
2.3 Les données : `CIFAR10Data` . . . 4
2.4 Les modèles : `ModelZoo` . . . 5
2.5 Le planning du learning rate : `OneCycleSchedule` . . . 5
2.6 L'entraînement : `Trainer` . . . 5
2.7 L'évaluation et les résultats : `Evaluator` . . . 6
2.8 L'orchestration : `Experiment` . . . 6
2.9 Lancement en ligne de commande . . . 6

3 Préparation des données **7**
3.1 Chargement et découpage . . . 7
3.2 Normalisation . . . 7
3.3 Sous-échantillon équilibré pour le benchmark . . . 7
3.4 Data augmentation . . . 7
3.5 Pipeline `tf.data` . . . 8
3.6 Autres techniques de régularisation et d'entraînement . . . 8

4 Implémentation et résultats des cinq modèles **8**
4.1 Démarche en entonnoir : pourquoi tous les modèles ne vont pas en phase finale . . . 9
4.2 MLP (perceptron multicouche) . . . 10
4.3 CNN simple . . . 11
4.4 VGG-BN . . . 12
4.5 ResNet-9 . . . 13
4.6 Modèle personnalisé (`home_made`) . . . 14

5 Conclusion et analyse globale **16**
5.1 Synthèse comparative . . . 16
5.2 Analyse globale . . . 16
5.3 Limites . . . 17
5.4 Pistes d'amélioration . . . 18
5.5 Conclusion . . . 18

A Annexe graphique **20**
A.1 Phase 1 – Benchmark (10 000 images, 15 epochs) 20
A.2 Phase 2 – Tuning du VGG-BN 22
A.3 Phase 3 – Modèles finaux (45 000 images, 40 epochs) 23
A.4 Tableau complet des résultats 29

1 Introduction du projet, jeu de données et objectifs

1.1 Contexte

Ce projet s'inscrit dans le module de Deep Learning du Mastère IA. Le sujet demande d'obtenir *la meilleure performance possible* sur le jeu de données CIFAR-10, en testant plusieurs modèles et en analysant finement leurs résultats. Le travail est rendu sous la forme d'un script Python unique, organisé en programmation orientée objet, accompagné du présent rapport.

1.2 Le jeu de données CIFAR-10

CIFAR-10 est un jeu de données d'imagettes en couleur d'objets du quotidien. Ses caractéristiques sont les suivantes :

— images de taille 32×32 pixels, en 3 canaux (RGB) ;
— 10 classes, équilibrées ;
— 50 000 images d'apprentissage et 10 000 images de test.

Indice	**Classe**	**Indice**	**Classe**
0	airplane	5	dog
1	automobile	6	frog
2	bird	7	horse
3	cat	8	ship
4	deer	9	truck

TABLE 1 – Les 10 classes de CIFAR-10 (liste `CLASS_NAMES` du code).

La très faible résolution des images et la ressemblance entre certaines classes (chat/chien, camion/automobile, cerf/cheval) rendent la tâche difficile pour des modèles qui ne tirent pas parti de la structure spatiale de l'image.

1.3 Découpage des données

Le jeu d'apprentissage officiel est séparé en deux parties : 45 000 images pour l'entraînement et 5 000 pour la validation. Le jeu de test (10 000 images) n'est utilisé que pour l'évaluation finale.

Ensemble	**Images**	**Usage**
Entraînement	45 000	apprentissage des poids
Validation	5 000	choix du meilleur epoch et des hyperparamètres
Test	10 000	évaluation finale

TABLE 2 – Découpage des données.

1.4 Objectifs

Les objectifs du projet sont :

1. **Benchmark** : comparer plusieurs modèles, du plus simple au plus profond, sur un sous-ensemble équilibré du jeu d'apprentissage (10 000 images, 15 epochs) ;
2. **Optimisation** : régler les hyperparamètres du meilleur modèle du benchmark (learning rate maximal, étude d'ablation de la data augmentation et du *label smoothing*) ;
3. **Confirmation** : ré-entraîner les modèles retenus sur le jeu complet (45 000 images, 40 epochs) et les évaluer finement (accuracy, *Test-Time Augmentation*, rapport par classe, matrice de confusion, erreurs les plus confiantes) ;

4. **Analyse** : interpréter les résultats, identifier les limites de la démarche et proposer des pistes d'amélioration.

Cinq modèles sont étudiés : un perceptron multicouche (`mlp`), un CNN simple (`simple_cnn`), un VGG compact avec BatchNorm (`vgg_bn`), un ResNet-9 (`resnet9`) et un modèle personnalisé (`home_made`). Tous ne parcourent pas les trois phases : la démarche « en entonnoir » adoptée et sa justification sont détaillées en section 4.1.

2 Structure générale du code

2.1 Vue d'ensemble

Le code est un script unique (`cifar10_challenge.py`) exécutable en ligne de commande. Il est organisé en classes ayant chacune une responsabilité précise.

Élément	Rôle
`TrainConfig`	*dataclass* regroupant les hyperparamètres d'un entraînement
`CIFAR10Data`	chargement, découpage, normalisation, sous-échantillonnage, augmentation et pipelines `tf.data`
`ModelZoo`	catalogue des 5 architectures et registre des paramètres par défaut de chacune
`OneCycleSchedule`	planning du learning rate (montée linéaire puis décroissance cosinus)
`Trainer`	entraînement d'un modèle (optimiseur, perte, callbacks, sauvegarde des poids)
`Evaluator`	évaluation sur le test (accuracy, TTA, top-2, rapport par classe) et tracé des figures
`Experiment`	orchestration des phases benchmark, tuning et final et sauvegarde des résultats
`setup_hardware`, `parse_args`, `main`	configuration GPU / précision mixte, arguments en ligne de commande, point d'entrée

TABLE 3 – Classes et fonctions principales du script.

2.2 La configuration : `TrainConfig`

Chaque entraînement est décrit par une instance de `TrainConfig`, ce qui garantit que tous les paramètres sont tracés et reproductibles.

```python
@dataclass
class TrainConfig:
    model_name: str = "resnet9"
    epochs: int = 30
    batch_size: int = 128
    max_lr: float = 0.1 # LR max du One-Cycle (SGD)
    momentum: float = 0.9
    weight_decay: float = 5e-4 # regularisation L2
    label_smoothing: float = 0.1
    augment: str = "full" # "none" | "basic" | "full"
    warmup_pct: float = 0.2 # part des iterations en montee du LR
    tag: str = "" # etiquette libre (phase / variante)
```

2.3 Les données : `CIFAR10Data`

Cette classe charge CIFAR-10 via Keras, crée la séparation entraînement/validation, calcule les statistiques de normalisation, fournit un sous-échantillon équilibré (`subset`) et construit les pipelines `tf.data` avec augmentation. Elle est détaillée dans la partie 3.

2.4 Les modèles : `ModelZoo`

`ModelZoo` contient une méthode de classe par architecture (`mlp`, `simple_cnn`, `vgg_bn`, `resnet9`, `home_made`), deux fonctions utilitaires partagées (`_conv_bn_relu` et `_head`) et un dictionnaire `REGISTRY` associant à chaque modèle son learning rate maximal et son niveau d'augmentation par défaut. La méthode `build(name, wd)` instancie un modèle à partir de son nom.

Modèle	`max_lr`	`augment`
`mlp`	0,05	none
`simple_cnn`	0,05	basic
`vgg_bn`	0,1	full
`resnet9`	0,1	full
`home_made`	0,05	basic

TABLE 4 – Paramètres par défaut du registre `REGISTRY`.

Tous les modèles se terminent par la même tête de classification : une couche `Dense(10)` suivie d'un `softmax` forcé en `float32` pour rester stable en précision mixte.

2.5 Le planning du learning rate : `OneCycleSchedule`

Le learning rate suit une politique *One-Cycle* : une montée linéaire pendant les premiers 20 % des itérations, puis une décroissance en cosinus. Avec T le nombre total d'itérations, T_w celui de la phase de montée, $\eta_{\max}$ le learning rate maximal, $\eta_0 = \eta_{\max}/25$ et $\eta_f = \eta_{\max}/10^4$:

$$\eta(t) = \begin{cases} \eta_0 + (\eta_{\max} - \eta_0)\,\dfrac{t}{T_w} & \text{si } t < T_w, \\ \eta_f + \dfrac{1}{2}(\eta_{\max} - \eta_f)\left(1 + \cos\left(\pi\,\dfrac{t - T_w}{T - T_w}\right)\right) & \text{sinon.} \end{cases}$$

Cette stratégie permet de converger rapidement et de réduire la sensibilité au choix du learning rate.

2.6 L'entraînement : `Trainer`

La méthode `train` reçoit une `TrainConfig` et un jeu d'apprentissage, puis :

1. construit les datasets d'entraînement (avec augmentation) et de validation ;
2. instancie le modèle via `ModelZoo.build` ;
3. configure l'optimiseur SGD avec momentum de Nesterov et le planning `OneCycleSchedule` ;
4. compile le modèle avec une entropie croisée catégorielle et du *label smoothing* ;
5. lance `model.fit` avec deux callbacks : `ModelCheckpoint` (sauvegarde des poids du meilleur epoch selon `val_accuracy`) et `CSVLogger` (historique de l'entraînement) ;
6. recharge les poids du meilleur epoch et renvoie le modèle, l'historique, le temps d'entraînement et le nombre de paramètres.

```
optimizer = keras.optimizers.SGD(
    learning_rate=OneCycleSchedule(cfg.max_lr, steps, cfg.warmup_pct),
    momentum=cfg.momentum, nesterov=True)
model.compile(optimizer=optimizer,
              loss=keras.losses.CategoricalCrossentropy(
                  label_smoothing=cfg.label_smoothing),
              metrics=["accuracy"])
```

2.7 L'évaluation et les résultats : `Evaluator`

Cette classe évalue un modèle sur le jeu de test et produit les figures. Pour chaque modèle, elle calcule :

- l'*accuracy* de test simple ;
- l'*accuracy* avec *Test-Time Augmentation* (TTA), qui moyenne les prédictions de l'image et de son miroir horizontal ;
- l'accuracy *top-2* ;
- en mode détaillé : rapport de classification par classe, matrice de confusion normalisée et grille des 25 erreurs les plus confiantes.

Les méthodes `plot_history`, `plot_confusion`, `plot_misclassified`, `plot_comparison` et `plot_val_curves` enregistrent leurs figures dans `results_cifar10/figures/`. Le fichier `<run>_history.png` contient les courbes de loss et d'accuracy par epoch, utilisées dans la partie 4. L'ensemble des figures complémentaires est regroupé dans l'annexe graphique A.

2.8 L'orchestration : `Experiment`

La classe `Experiment` enchaîne les trois phases du sujet. Chaque exécution est stockée dans un dictionnaire de résultats (modèle, nombre d'images, epochs, learning rate, augmentation, paramètres, temps d'entraînement, meilleure accuracy de validation, accuracies de test, écart d'overfitting) puis sauvegardée dans `results_summary.csv` et `results_full.json`.

Phase	Méthode	Contenu
1. Benchmark	`benchmark()`	entraîne les modèles sur un sous-ensemble équilibré (15 epochs) et retient le meilleur sur la validation
2. Tuning	`tune()`	recherche du learning rate max (grille 0,05 / 0,1 / 0,2), puis ablation : sans augmentation, augmentation basique, augmentation complète, augmentation complète sans label smoothing
3. Final	`final()`	entraînement sur les 45 000 images (40 epochs) et évaluation détaillée

TABLE 5 – Les trois phases orchestrées par `Experiment`.

2.9 Lancement en ligne de commande

Le script est piloté par `argparse`. L'option `-home-made` lance uniquement le modèle personnalisé sur le jeu complet et `-quick` permet un test de quelques minutes.

```
python cifar10_challenge.py # pipeline complet
python cifar10_challenge.py --phase benchmark # benchmark seul
python cifar10_challenge.py --phase final --final-models resnet9 vgg_bn
python cifar10_challenge.py --home-made # modele personnalise seul
python cifar10_challenge.py --quick # test rapide
```

Enfin, `setup_hardware` active la croissance mémoire des GPU et la précision mixte (`mixed_float16`) lorsqu'un GPU est disponible et `keras.utils.set_random_seed` fixe la graine pour la reproductibilité.

3 Préparation des données

3.1 Chargement et découpage

Les données sont chargées avec `keras.datasets.cifar10.load_data()`. Les 50 000 images d'apprentissage sont mélangées par une permutation aléatoire à graine fixée (`seed=42`), puis 5 000 images sont réservées à la validation. Ce découpage est aléatoire (et non strictement stratifié), mais le nombre d'images par classe reste proche de l'équilibre grâce à la taille de l'échantillon.

```
rng = np.random.default_rng(seed)
idx = rng.permutation(len(x_train))
val_idx, tr_idx = idx[:val_size], idx[val_size:]
self.x_train, self.y_train = x_train[tr_idx], y_train[tr_idx]
self.x_val, self.y_val = x_train[val_idx], y_train[val_idx]
```

3.2 Normalisation

Les pixels sont mis à l'échelle dans $[0, 1]$ puis centrés-réduits canal par canal :

$$\tilde{x}_c = \frac{x_c/255 - \mu_c}{\sigma_c}, \qquad c \in \{R, G, B\}.$$

La moyenne μ_c et l'écart-type σ_c sont calculés **uniquement sur l'ensemble d'entraînement**, afin d'éviter toute fuite d'information depuis la validation ou le test. Ces mêmes statistiques sont ensuite appliquées aux trois ensembles.

3.3 Sous-échantillon équilibré pour le benchmark

Pour comparer rapidement les modèles, la méthode `subset(n)` extrait $n/10$ images par classe (10 000 images au total), tirées aléatoirement sans remise, puis mélangées.

```
per_class = n // NUM_CLASSES
idx = np.concatenate([
    rng.choice(np.where(self.y_train == c)[0], per_class, replace=False)
    for c in range(NUM_CLASSES)])
rng.shuffle(idx)
```

3.4 Data augmentation

L'augmentation de données crée artificiellement de la diversité à chaque epoch, ce qui réduit le surapprentissage. Elle est appliquée *uniquement à l'entraînement*, après la normalisation. Trois niveaux sont disponibles :

Niveau	Transformations
`none`	aucune
`basic`	padding de 4 pixels (mode *reflect*), recadrage aléatoire 32×32, retournement horizontal aléatoire
`full`	`basic` + *Cutout* (masquage d'un carré aléatoire 8×8)

TABLE 6 – Niveaux d'augmentation de données.

Recadrage et retournement. Le recadrage aléatoire décale légèrement l'objet dans l'image et le miroir horizontal double virtuellement le nombre de configurations. Ces deux transformations préservent le sens de l'image (un cheval retourné reste un cheval).

Cutout. Un carré de 8×8 pixels est mis à zéro, ce qui correspond à la valeur moyenne après normalisation. Le modèle ne peut plus dépendre d'une seule zone de l'image et apprend des représentations plus robustes.

```
def _augment(self, img, level):
    if level in ("basic", "full"):
        img = tf.pad(img, [[4, 4], [4, 4], [0, 0]], mode="REFLECT")
        img = tf.image.random_crop(img, [32, 32, 3])
        img = tf.image.random_flip_left_right(img)
    if level == "full":
        img = self._cutout(img, 8)
    return img
```

3.5 Pipeline `tf.data`

La méthode `make_dataset` construit le pipeline : normalisation, augmentation (si entraînement), encodage *one-hot* des étiquettes, mélange à chaque epoch, découpage en batchs et `prefetch` pour recouvrir le calcul et le chargement des données.

```
def prep(img, label):
    img = (tf.cast(img, tf.float32) / 255.0 - mean) / std
    if training and augment != "none":
        img = self._augment(img, augment)
    return img, tf.one_hot(label, NUM_CLASSES)

ds = tf.data.Dataset.from_tensor_slices((x, y))
if training:
    ds = ds.shuffle(len(x), reshuffle_each_iteration=True)
ds = ds.map(prep, num_parallel_calls=AUTOTUNE).batch(batch_size).prefetch(AUTOTUNE)
```

3.6 Autres techniques de régularisation et d'entraînement

Outre l'augmentation, les entraînements utilisent :

- le **label smoothing** ($\varepsilon = 0{,}1$) : les étiquettes one-hot sont remplacées par $y' = (1-\varepsilon)\, y + \varepsilon/K$ avec $K = 10$, ce qui évite des prédictions trop confiantes ;
- la **régularisation L2** ($\lambda = 5 \cdot 10^{-4}$) sur les poids des couches ;
- le **Test-Time Augmentation** à l'évaluation : moyenne des probabilités prédites pour l'image originale et son miroir horizontal.

4 Implémentation et résultats des cinq modèles

Protocole commun

Tous les modèles sont entraînés avec le même protocole : SGD avec momentum de Nesterov (0,9), planning One-Cycle, régularisation L2 ($5 \cdot 10^{-4}$), label smoothing (0,1), taille de batch de 128 et sélection du meilleur epoch sur la validation. Seuls le learning rate maximal et le niveau d'augmentation varient d'un modèle à l'autre. Les scores sont des accuracies sur le jeu de **test** (10 000 images).

Attention : tous les modèles n'ont pas été entraînés dans les mêmes conditions. Le MLP et le CNN simple ont été évalués qu'en phase de benchmark (10 000 images, 15 epochs), alors que le VGG-BN, le ResNet-9 et le modèle personnalisé l'ont été en phase finale (45 000 images, 40 epochs). La table 7 précise les conditions de chaque score et la section 4.1 justifie ce choix.

Modèle	Phase	Images	Epochs	max_lr	Augm.	Paramètres
mlp	benchmark	10 000	15	0,05	none	3 682 826
simple_cnn	benchmark	10 000	15	0,05	basic	1 116 970
vgg_bn	final	45 000	40	0,2	none	1 217 226
resnet9	final	45 000	40	0,1	full	6 577 610
home_made	final	45 000	40	0,05	basic	258 217

TABLE 7 – Conditions d'entraînement de chaque score (d'après `results_summary.csv`). Pour le VGG-BN, `max_lr` et l'augmentation sont ceux retenus par la phase de tuning (valeurs par défaut du registre : 0,1 et `full`).

4.1 Démarche en entonnoir : pourquoi tous les modèles ne vont pas en phase finale

Entraîner chaque modèle sur le jeu complet, avec un réglage d'hyperparamètres propre, aurait été très coûteux. La démarche suit donc un **entonnoir** : on commence par comparer largement et à bas coût, puis on concentre le budget de calcul sur un nombre de plus en plus réduit de modèles.

Phase	Modèles	Données / epochs	Critère de passage	Temps cumulé
1. Benchmark	4 : `mlp`, `simple_cnn`, `vgg_bn`, `resnet9`	10 000 img. / 15	meilleure accuracy de **validation**	1 609 s (≈ 27 min)
2. Tuning	1 : `vgg_bn` (7 entraînements)	10 000 img. / 15	meilleure accuracy de **validation**	2 432 s (≈ 41 min)
3. Final	2 : `vgg_bn`, `resnet9`	45 000 img. / 40	accuracy de test avec TTA	13 140 s (≈ 3 h 39)
Hors entonnoir	1 : `home_made` (option `-home-made`)	45 000 img. / 40	–	1 476 s (≈ 25 min)

TABLE 8 – Entonnoir expérimental et temps d'entraînement mesurés (colonne `train_time_s`). Total : environ 18 660 s, soit ≈ 5 h 10.

1. Un benchmark réduit pour comparer vite. Entraîner un modèle sur 10 000 images pendant 15 epochs représente 150 000 images vues, contre 1 800 000 en phase finale, soit 12 fois moins. Pour le ResNet-9, cela correspond à 1 140 s contre 9 104 s. Cette phase permet d'écarter rapidement les modèles les moins prometteurs, de choisir sur quel modèle porter l'effort de réglage et de le faire sur la **validation** et non sur le test, pour ne pas biaiser la sélection.

2. Pourquoi le MLP et le CNN simple n'atteignent pas la phase finale ? Dans les conditions du benchmark, ils sont nettement en retrait : 51,74 % et 68,34 % de test avec TTA, contre 71,58 % pour le VGG-BN. Le MLP ignore la structure spatiale des images et le CNN simple n'a ni BatchNorm ni augmentation forte. Les deux approches jouent le rôle de *références de base* (*baselines*) permettant de mesurer l'apport de la convolution, puis de la normalisation, plutôt que celui de candidats au meilleur score. Il faut toutefois noter que leur coût était faible (24 s et 58 s en benchmark) : leur exclusion de la phase finale est un **choix de périmètre** et non une contrainte de calcul. C'est une limite assumée (voir section 5.3).

3. Pourquoi le tuning ne porte que sur un seul modèle. Le tuning comprend 7 entraînements (3 learning rates, puis 4 variantes d'augmentation / label smoothing). Sur le VGG-BN, il a coûté environ 41 min. Le même plan appliqué au ResNet-9 aurait coûté environ 2 h 13 et appliqué aux quatre modèles du benchmark environ 3 h 08 (estimations à partir des temps du benchmark). Le réglage est donc réservé au meilleur modèle du benchmark selon la validation, ici le VGG-BN (71,50 % de validation, contre 66,58 % pour le CNN simple et 64,58 % pour le ResNet-9). Les hyperparamètres retenus ($\eta_{\max} = 0{,}2$, sans augmentation) ne sont ensuite appliqués qu'à ce modèle.

4. Pourquoi le ResNet-9 est conservé en phase finale malgré son résultat de benchmark. Le ResNet-9 est le plus grand modèle (6,58 M de paramètres, soit 5,4 fois plus que le VGG-BN) et utilise l'augmentation `full`, qui ralentit l'apprentissage. Un benchmark de 15 epochs sur 10 000 images est défavorable à ce type de modèle : il est classé dernier des modèles convolutifs (64,58 % en validation) alors que l'architecture résiduelle est reconnue pour sa performance sur CIFAR-10. Il a donc été gardé comme second finaliste (c'est aussi la valeur par défaut de l'option `-final-models`) afin de tester cette hypothèse sur le jeu complet. Le résultat la confirme : 94,03 % de test contre 90,30 % pour le VGG-BN. Une sélection fondée uniquement sur le benchmark réduit aurait donc écarté le meilleur modèle du projet. Le même raisonnement aurait pu justifier de garder le CNN simple, ce qui souligne la fragilité d'un critère de sélection établi sur un petit sous-ensemble.

5. Le modèle personnalisé, hors entonnoir. Le `home_made` est lancé séparément avec l'option `-home-made`, directement en phase finale et avec les paramètres par défaut du registre ($\eta_{\max} = 0{,}05$, augmentation `basic`). Il n'a donc pas été comparé aux autres en benchmark, ni réglé. Il est évalué pour son compromis entre performance, taille (0,26 M de paramètres) et temps de calcul et non pour concurrencer les modèles réglés.

6. Sélection du modèle final. À l'issue de la phase 3, le meilleur modèle est désigné selon l'accuracy de test avec TTA (ResNet-9). Les phases 1 et 2 sélectionnent sur la validation ; cette dernière étape devrait idéalement faire de même (voir limites).

4.2 MLP (perceptron multicouche)

Principe. Le MLP est la référence « sans convolution » : l'image est aplatie en un vecteur de 3 072 valeurs, puis traitée par deux couches denses (1024 et 512 neurones), chacune suivie d'une BatchNorm, d'une ReLU et d'un Dropout de 0,3. Ce modèle ignore la structure spatiale de l'image (voisinage des pixels, invariance par translation), ce qui limite fortement sa capacité sur des images.

```
@classmethod
def mlp(cls, wd):
    inp = keras.Input((32, 32, 3))
    x = layers.Flatten()(inp)
    for units in (1024, 512):
        x = layers.Dense(units, kernel_regularizer=regularizers.l2(wd))(x)
        x = layers.BatchNormalization()(x)
        x = layers.Activation("relu")(x)
        x = layers.Dropout(0.3)(x)
    return keras.Model(inp, cls._head(x, wd), name="mlp")
```

Score (phase benchmark : 10 000 images, 15 epochs).

Val. acc. (%)	**Test acc.** (%)	**Test acc. + TTA** (%)	**Temps** (s)
50,42	50,24	51,74	24

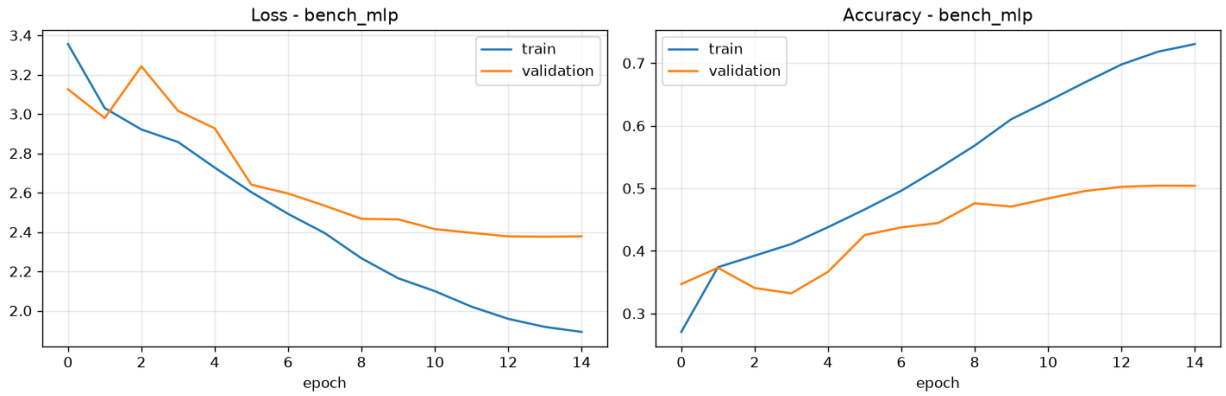


FIGURE 1 – Évolution de la loss et de l'accuracy par epoch – MLP.

Courbes d'apprentissage.

4.3 CNN simple

Principe. Le `simple_cnn` est un réseau convolutif classique, sans BatchNorm : deux blocs de deux convolutions 3×3 (32 puis 64 filtres) suivis d'un *max pooling* et d'un Dropout de 0,25, puis une couche dense de 256 neurones avec Dropout de 0,5. Les convolutions exploitent la structure locale de l'image et partagent leurs poids, ce qui réduit le nombre de paramètres et apporte une invariance par translation.

```
@classmethod
def simple_cnn(cls, wd):
    inp = keras.Input((32, 32, 3))
    x = inp
    for f in (32, 64):
        x = layers.Conv2D(f, 3, padding="same", activation="relu",
                          kernel_regularizer=regularizers.l2(wd))(x)
        x = layers.Conv2D(f, 3, padding="same", activation="relu",
                          kernel_regularizer=regularizers.l2(wd))(x)
        x = layers.MaxPooling2D()(x)
        x = layers.Dropout(0.25)(x)
    x = layers.Flatten()(x)
    x = layers.Dense(256, activation="relu",
                     kernel_regularizer=regularizers.l2(wd))(x)
    x = layers.Dropout(0.5)(x)
    return keras.Model(inp, cls._head(x, wd), name="simple_cnn")
```

Score (phase benchmark : 10 000 images, 15 epochs).

Val. acc. (%)	**Test acc.** (%)	**Test acc. + TTA** (%)	**Temps** (s)
66,58	68,12	68,34	58

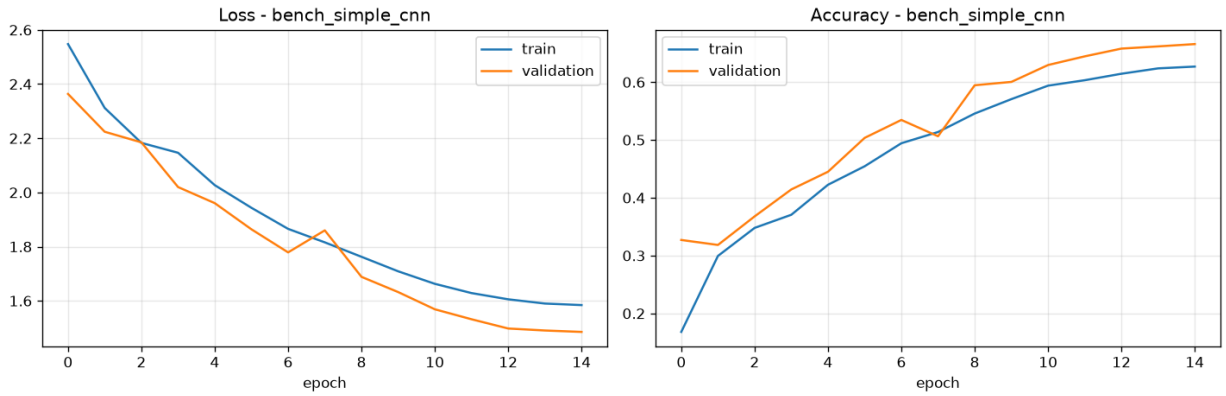


FIGURE 2 – Évolution de la loss et de l'accuracy par epoch – CNN simple.

Courbes d'apprentissage.

4.4 VGG-BN

Principe. Le `vgg_bn` est une version compacte de l'architecture VGG : trois blocs de deux convolutions 3×3 (64, 128 puis 256 filtres), chaque convolution étant suivie d'une BatchNorm et d'une ReLU. Chaque bloc se termine par un max pooling et un Dropout progressif (0,2, 0,3, 0,4). Un *Global Average Pooling* remplace l'aplatissement, ce qui réduit fortement le nombre de paramètres, puis une couche dense de 256 neurones (avec BatchNorm et Dropout de 0,5) précède la sortie. La BatchNorm stabilise et accélère l'entraînement ; le Dropout progressif régularise davantage les couches profondes.

Configuration finale. Le VGG-BN a été le meilleur modèle du benchmark en validation, c'est pourquoi la phase de tuning a porté sur lui (section 4.1). Elle a retenu $\eta_{\max} = 0{,}2$ et **aucune augmentation** (`augment="none"`), meilleure variante sur 10 000 images et c'est cette configuration qui a été utilisée en phase finale. Elle explique le surapprentissage observé : 99,0 % d'accuracy d'entraînement contre 91,2 % en validation (écart de 7,8 points).

```
@classmethod
def vgg_bn(cls, wd):
    inp = keras.Input((32, 32, 3))
    x = inp
    for f, drop in ((64, 0.2), (128, 0.3), (256, 0.4)):
        x = cls._conv_bn_relu(x, f, wd)
        x = cls._conv_bn_relu(x, f, wd)
        x = layers.MaxPooling2D()(x)
        x = layers.Dropout(drop)(x)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dense(256, use_bias=False,
                     kernel_regularizer=regularizers.l2(wd))(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.Dropout(0.5)(x)
    return keras.Model(inp, cls._head(x, wd), name="vgg_bn")
```

Score (phase finale : 45 000 images, 40 epochs).

Val. acc. (%)	**Test acc.** (%)	**Test acc. + TTA** (%)	**Temps** (s)
91,18	90,30	90,97	4 035

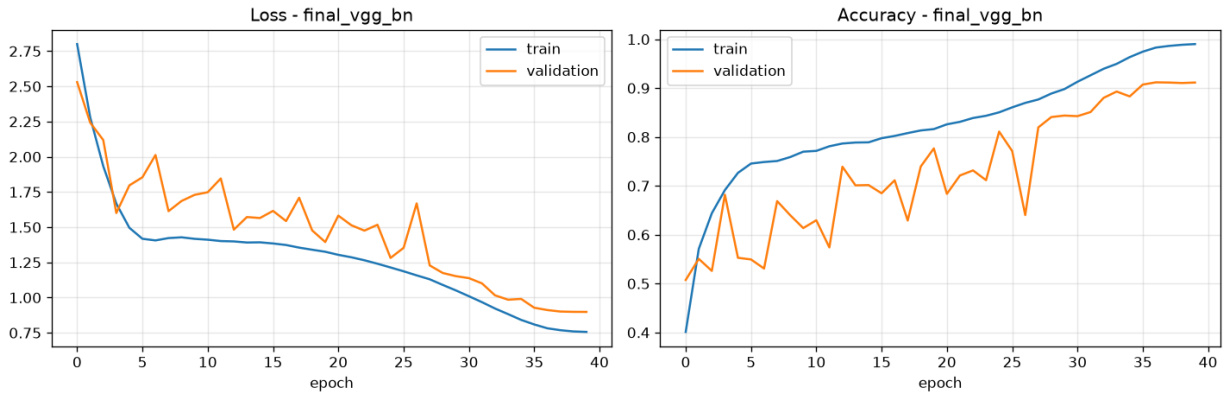


FIGURE 3 – Évolution de la loss et de l'accuracy par epoch – VGG-BN.

Courbes d'apprentissage.

4.5 ResNet-9

Principe. Le `resnet9` est un réseau résiduel à 9 couches pondérées, rendu célèbre par le benchmark DAWNBench. Il alterne des blocs convolutifs (Conv–BatchNorm–ReLU) avec des max pooling et comporte deux **blocs résiduels** : la sortie d'un bloc de deux convolutions est *ajoutée* à son entrée ($y = x + F(x)$). Ces connexions de raccourci facilitent la propagation du gradient et permettent d'entraîner un réseau plus profond et plus large (jusqu'à 512 filtres) avec une convergence rapide. Un *Global Max Pooling* et un Dropout de 0,2 précèdent la sortie. Ce modèle a été entraîné avec ses paramètres par défaut ($\eta_{\max} = 0{,}1$, augmentation `full`).

```
@classmethod
def resnet9(cls, wd):
    cbr = lambda t, f: cls._conv_bn_relu(t, f, wd)
    inp = keras.Input((32, 32, 3))
    x = cbr(inp, 64) # 32x32
    x = cbr(x, 128)
    x = layers.MaxPooling2D()(x) # 16x16
    r = cbr(cbr(x, 128), 128)
    x = layers.Add()([x, r]) # bloc residuel 1
    x = cbr(x, 256)
    x = layers.MaxPooling2D()(x) # 8x8
    x = cbr(x, 512)
    x = layers.MaxPooling2D()(x) # 4x4
    r = cbr(cbr(x, 512), 512)
    x = layers.Add()([x, r]) # bloc residuel 2
    x = layers.GlobalMaxPooling2D()(x)
    x = layers.Dropout(0.2)(x)
    return keras.Model(inp, cls._head(x, wd), name="resnet9")
```

Score (phase finale : 45 000 images, 40 epochs).

Val. acc. (%)	**Test acc.** (%)	**Test acc. + TTA** (%)	**Temps** (s)
93,92	94,03	94,45	9 104

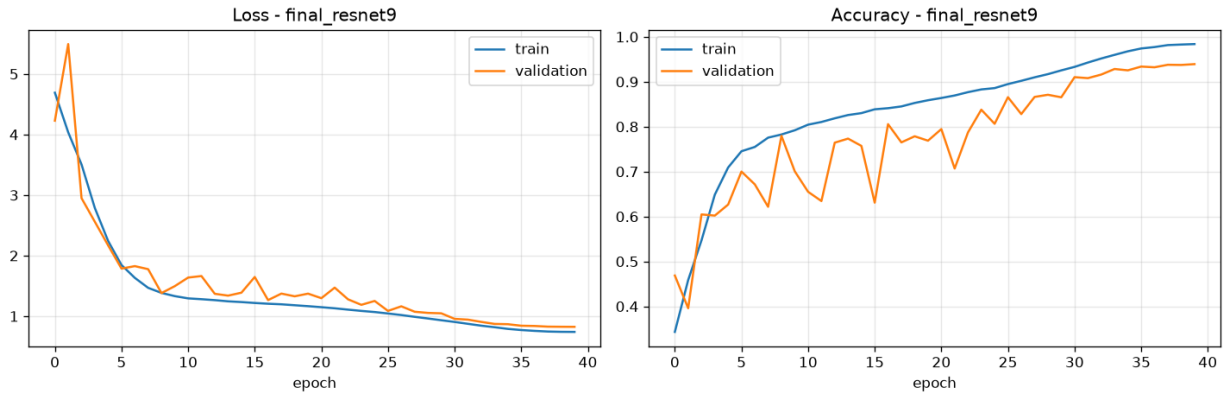


FIGURE 4 – Évolution de la loss et de l'accuracy par epoch – ResNet-9.

Courbes d'apprentissage.

4.6 Modèle personnalisé (`home_made`)

Principe. Le `home_made` est un petit CNN conçu par nos soins : trois convolutions 3×3 de filtres décroissants (75, 50 puis 25), chacune suivie d'une BatchNorm et d'un max pooling, avec un Dropout de 0,2 après la deuxième convolution. Les cartes de caractéristiques ($4 \times 4 \times 25$) sont aplaties et passent par une couche dense de 512 neurones avec un Dropout de 0,3. Ce modèle est volontairement léger (environ 0,26 M de paramètres), avec une augmentation de niveau `basic`. Il se lance seul grâce à l'option `-home-made`.

```
@classmethod
def home_made(cls, wd):
    inp = keras.Input((32, 32, 3))
    x = layers.Conv2D(75, (3, 3), strides=1, padding="same", activation="relu",
                      kernel_regularizer=regularizers.l2(wd))(inp)
    x = layers.BatchNormalization()(x)
    x = layers.MaxPooling2D((2, 2), strides=2, padding="same")(x)
    x = layers.Conv2D(50, (3, 3), strides=1, padding="same", activation="relu",
                      kernel_regularizer=regularizers.l2(wd))(x)
    x = layers.Dropout(0.2)(x)
    x = layers.BatchNormalization()(x)
    x = layers.MaxPooling2D((2, 2), strides=2, padding="same")(x)
    x = layers.Conv2D(25, (3, 3), strides=1, padding="same", activation="relu",
                      kernel_regularizer=regularizers.l2(wd))(x)
    x = layers.BatchNormalization()(x)
    x = layers.MaxPooling2D((2, 2), strides=2, padding="same")(x)
    x = layers.Flatten()(x)
    x = layers.Dense(512, activation="relu",
                     kernel_regularizer=regularizers.l2(wd))(x)
    x = layers.Dropout(0.3)(x)
    return keras.Model(inp, cls._head(x, wd), name="home_made")
```

Score (phase finale : 45 000 images, 40 epochs).

Val. acc. (%)	**Test acc.** (%)	**Test acc. + TTA** (%)	**Temps** (s)
85,92	85,38	86,23	1 476

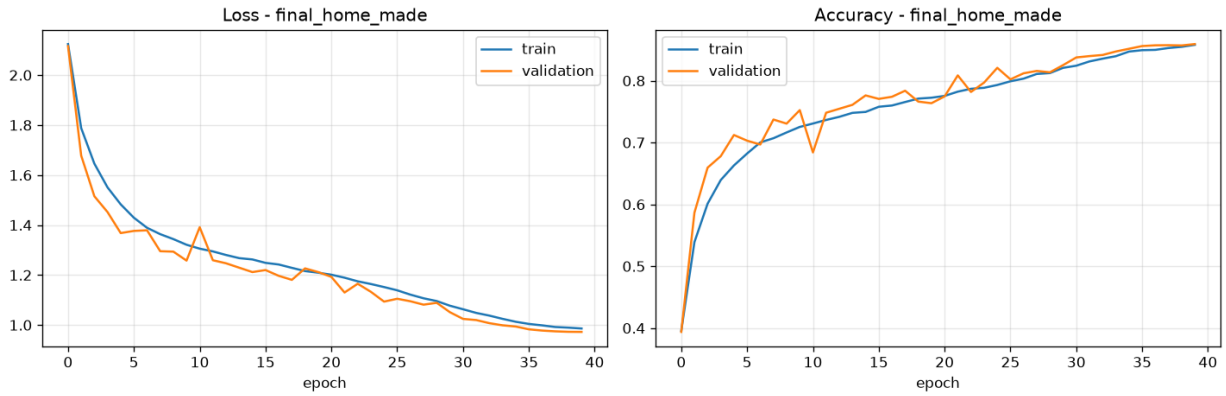


FIGURE 5 – Évolution de la loss et de l'accuracy par epoch – modèle personnalisé.

Courbes d'apprentissage.

5 Conclusion et analyse globale

5.1 Synthèse comparative

Modèle	Params	Phase	Val. (%)	Test (%)	Test + TTA (%)	Temps (s)
MLP	3,68 M	benchmark	50,42	50,24	51,74	24
CNN simple	1,12 M	benchmark	66,58	68,12	68,34	58
VGG-BN	1,22 M	final	91,18	90,30	90,97	4 035
ResNet-9	6,58 M	final	93,92	94,03	94,45	9 104
Home-made	0,26 M	final	85,92	85,38	86,23	1 476

TABLE 9 – Comparaison des cinq modèles. *Benchmark* : 10 000 images, 15 epochs ; *final* : 45 000 images, 40 epochs. Les scores des deux groupes ne sont donc pas directement comparables.

5.2 Analyse globale

Apport de la convolution. Le MLP traite chaque pixel indépendamment de sa position. Les modèles convolutifs exploitent la localité et le partage de poids, ce qui leur permet de mieux généraliser avec moins de paramètres. Dans les mêmes conditions de benchmark, le CNN simple dépasse le MLP de 16,6 points (68,34 % contre 51,74 % avec TTA) avec 3,3 fois moins de paramètres (1,12 M contre 3,68 M). Le MLP sur-apprend nettement : 73,0 % en entraînement contre 50,4 % en validation (écart de 22,6 points), alors que le CNN simple ne présente aucun écart positif (62,7 % contre 66,6 %).

Apport de la BatchNorm et de la profondeur. Le VGG-BN ajoute la normalisation par batch, le Dropout progressif et une tête en Global Average Pooling. Le ResNet-9 ajoute en plus des connexions résiduelles et des couches plus larges. Dans les mêmes conditions de benchmark, le VGG-BN dépasse le CNN simple de 3,2 points avec TTA (71,58 % contre 68,34 %) pour un nombre de paramètres comparable (1,22 M contre 1,12 M), alors que le ResNet-9 est en retrait (65,27 %). Le saut vers 90–94 % en phase finale provient surtout de l'entraînement sur 4,5 fois plus de données et 2,7 fois plus d'epochs, combiné à des réglages différents. Sans avoir réentraîné le CNN simple sur le jeu complet, il est impossible d'isoler la part due à l'architecture elle-même.

Un classement qui s'inverse entre benchmark et final. Sur 10 000 images, le VGG-BN devance le ResNet-9 (71,50 % contre 64,58 % en validation). Sur le jeu complet, c'est l'inverse : 94,03 % pour le ResNet-9 contre 90,30 % pour le VGG-BN sur le test. Le ResNet-9, plus grand (6,58 M de paramètres) et muni d'une augmentation `full`, a besoin de davantage de données et d'epochs pour s'exprimer. Cela illustre la limite d'un benchmark réalisé sur un sous-ensemble et justifie a posteriori de l'avoir conservé en phase finale (section 4.1).

Modèle personnalisé. Avec 0,26 M de paramètres, le `home_made` compte environ 25 fois moins de paramètres que le ResNet-9 et 4,7 fois moins que le VGG-BN. Il atteint 86,23 % avec TTA en 1 476 s, contre 90,97 % en 4 035 s pour le VGG-BN et 94,45 % en 9 104 s pour le ResNet-9. Il perd donc 4,7 à 8,2 points d'accuracy, mais il est entraîné 2,7 à 6,2 fois plus vite et s'avère le plus économe en paramètres : c'est le meilleur compromis taille / temps de calcul. Il n'a toutefois été ni réglé ni comparé aux autres en phase de benchmark.

Variante	Val. (%)	Test (%)	Écart train–val (pts)
Sans augmentation	76,98	76,40	+9,1
Augmentation `basic`	74,30	74,05	+1,1
Augmentation `full`	73,60	73,06	−0,8
`full`, sans label smoothing	73,58	73,10	−1,0

TABLE 10 – Résultats de l'étude d'ablation (phase `tune`).

Effet de l'augmentation et du label smoothing (ablation sur le VGG-BN, 10 000 images, 15 epochs, $\eta_{\max} = 0{,}2$). Sur un petit jeu de données entraîné pendant 15 epochs seulement, l'absence d'augmentation donne le meilleur score : une augmentation forte ralentit l'apprentissage et conduit à du sous-apprentissage (écart train–validation négatif). Le label smoothing a un effet négligeable (73,60 % contre 73,58 % en validation). Le learning rate maximal de 0,2 s'est aussi révélé meilleur que 0,1 et 0,05 (74,44 % contre 72,88 % et 69,84 % en validation, avec augmentation `full`).

Variabilité entre exécutions. La configuration « VGG-BN, $\eta_{\max} = 0{,}2$, augmentation `full`, label smoothing 0,1 » a été entraînée deux fois (recherche du learning rate, puis ablation) avec des résultats légèrement différents : 74,44 % puis 73,60 % en validation (73,84 % puis 73,06 % en test). Un écart d'environ 0,8 point apparaît donc sans aucun changement de configuration. Les différences inférieures à 1 point, comme l'effet du label smoothing, ne sont pas significatives ; en revanche, l'écart de 3,4 points entre « sans augmentation » et « augmentation `full` » dépasse nettement cette variabilité.

5.3 Limites

- **Conditions d'entraînement différentes** : le MLP et le CNN n'ont été évalués qu'en benchmark (10 000 images, 15 epochs) alors que le VGG-BN et le ResNet-9 l'ont été en phase finale (45 000 images, 40 epochs). Les écarts de score mélangent donc l'effet de l'architecture et celui de la quantité de données.
- **Entonnoir fondé sur un petit sous-ensemble** : le classement du benchmark s'est inversé en phase finale (VGG-BN meilleur en benchmark, ResNet-9 meilleur sur le jeu complet). Un critère d'élimination appliqué strictement aurait écarté le meilleur modèle ; le même doute porte sur les modèles non retenus (MLP, CNN simple).
- **Une seule exécution par modèle** : sans répétition avec plusieurs graines, on ne peut pas estimer précisément la variance des scores. Les deux entraînements d'une même configuration en phase de tuning montrent déjà un écart d'environ 0,8 point.
- **Hyperparamètres non homogènes** : learning rate et augmentation diffèrent selon les modèles, ce qui mélange l'effet de l'architecture et celui du réglage.
- **Tuning limité** à un seul modèle (VGG-BN), à une grille de trois learning rates et à quelques variantes d'augmentation, sur 10 000 images. Le ResNet-9 n'a pas été réglé et le choix « sans augmentation » pour le VGG-BN, optimal sur 10 000 images, n'a pas été revalidé sur le jeu complet.
- **Modèle personnalisé hors entonnoir** : il n'a pas été évalué en benchmark ni réglé et n'est donc pas comparé aux autres dans les mêmes conditions.
- **Validation non stratifiée** : le découpage est aléatoire, sans garantie stricte d'équilibre des classes.
- **Sélection du meilleur modèle final sur le test** : dans la phase finale, le meilleur modèle est désigné d'après l'accuracy de test (TTA). Une sélection sur la validation serait plus rigoureuse.
- **TTA minimaliste** : seul le miroir horizontal est utilisé.
- **Ressources** : l'entraînement complet suppose un GPU. Le ResNet-9 final a demandé environ 2 h 30 (9 104 s) et l'ensemble des expériences environ 5 h 10.

5.4 Pistes d'amélioration

- **Homogénéiser le protocole** : ré-entraîner les cinq modèles en phase finale, avec les mêmes données et le même nombre d'epochs.
- **Élargir l'entonnoir** : conserver plus de modèles à chaque étape, ou comparer en benchmark sur davantage d'images et d'epochs pour que le classement soit plus représentatif du résultat final.
- **Régler le ResNet-9** (learning rate, augmentation) comme cela a été fait pour le VGG-BN et revalider l'augmentation sur le jeu complet.
- **Augmentations plus riches** : Mixup, CutMix, AutoAugment ou RandAugment.
- **Architectures plus profondes** : ResNet-18/34, Wide-ResNet, ou transfert d'un modèle pré-entraîné (*fine-tuning*).
- **Entraînement plus long** et moyenne mobile exponentielle des poids (EMA).
- **Recherche d'hyperparamètres plus systématique** : recherche aléatoire ou bayésienne sur le learning rate, le weight decay, le dropout et la taille de batch, pour tous les modèles.
- **Ensembles de modèles** : moyenne des prédictions de plusieurs réseaux (ResNet-9, VGG-BN, home-made).
- **TTA plus riche** : ajout de recadrages décalés.
- **Évaluation plus robuste** : validation croisée et répétition sur plusieurs graines avec moyenne et écart-type.

5.5 Conclusion

Ce projet a permis de mettre en place un pipeline complet et reproductible pour CIFAR-10 : benchmark, réglage des hyperparamètres, entraînement sur le jeu complet et évaluation fine, selon une démarche en entonnoir qui concentre le calcul sur les modèles les plus prometteurs. L'organisation en classes (`CIFAR10Data`, `ModelZoo`, `Trainer`, `Evaluator`, `Experiment`) rend le code modulaire et facile à étendre avec de nouveaux modèles. **Résultats.** Le meilleur modèle est le ResNet-9, qui atteint **94,03%** d'accuracy sur le test (**94,45%** avec TTA). Il devance le VGG-BN (90,30%, soit 90,97% avec TTA) et le modèle personnalisé (85,38%, soit 86,23% avec TTA), mais au prix d'un temps d'entraînement plus élevé (9 104 s contre 4 035 s et 1 476 s). Le MLP (51,74%) et le CNN simple (68,34%) n'ont été évalués qu'en phase de benchmark et servent de références de base. **Enseignements.** Premièrement, la convolution est décisive : en benchmark, le CNN simple dépasse le MLP de 16,6 points avec 3,3 fois moins de paramètres. Deuxièmement, l'effet des réglages dépend du régime d'entraînement. Sur 10 000 images et 15 epochs, l'absence d'augmentation donne le meilleur score (3,4 points de mieux que l'augmentation complète), alors que sur le jeu complet le VGG-BN entraîné sans augmentation sur-apprend (99,0% en entraînement contre 91,2% en validation). Troisièmement, le classement s'inverse entre benchmark et phase finale. L'avance du ResNet-9 s'explique à la fois par sa capacité et son besoin de davantage de données et par le fait qu'il bénéficie d'une augmentation plus riche que le VGG-BN : l'architecture seule ne peut donc pas être créditée de tout l'écart. **Analyse des erreurs.** Les matrices de confusion montrent que les erreurs se concentrent sur les classes d'animaux proches, en particulier chat et chien : environ 11%, 7% et 13% des chats sont prédits comme des chiens par le VGG-BN, le ResNet-9 et le modèle personnalisé. Les erreurs les plus confiantes concernent souvent des images ambiguës à cette résolution de 32×32 pixels et plusieurs d'entre elles sont communes à plusieurs modèles, ce qui suggère que le plafond de performance tient en partie à la difficulté intrinsèque de certaines images. **Compromis coût / performance.** Avec 0,26 M de paramètres, le modèle personnalisé est environ 25 fois plus léger et 6,2 fois plus rapide à entraîner que le ResNet-9, pour une perte de 8,2 points d'accuracy. Ses courbes d'apprentissage d'entraînement et de validation quasi superposées indiquent qu'il est limité par sa capacité plutôt que par le surapprentissage, ce qui laisse une marge de progression en augmentant sa taille. **Limites et perspectives.** Ces conclusions doivent être lues avec prudence : les modèles n'ont pas tous été entraînés dans les mêmes conditions, chaque modèle n'a été entraîné qu'une fois et la variabilité observée entre deux exécutions d'une même

configuration (0,8 à 1,4 point) est du même ordre que certains écarts étudiés. Les prochaines étapes consisteraient à homogénéiser le protocole pour les cinq modèles, à régler le ResNet-9 comme le VGG-BN, à répéter les entraînements sur plusieurs graines, à tester des augmentations plus riches (Mixup, CutMix, AutoAugment) et à combiner les modèles par ensemble. Ces pistes permettraient de préciser la contribution réelle de chaque élément et de gagner encore quelques points sur ce jeu de données.

A Annexe graphique

Cette annexe regroupe les figures générées par le script (dossier `results_cifar10/figures/`) et le tableau complet des résultats. Les figures à insérer doivent être copiées dans le dossier `figures/` du projet.

A.1 Phase 1 – Benchmark (10 000 images, 15 epochs)

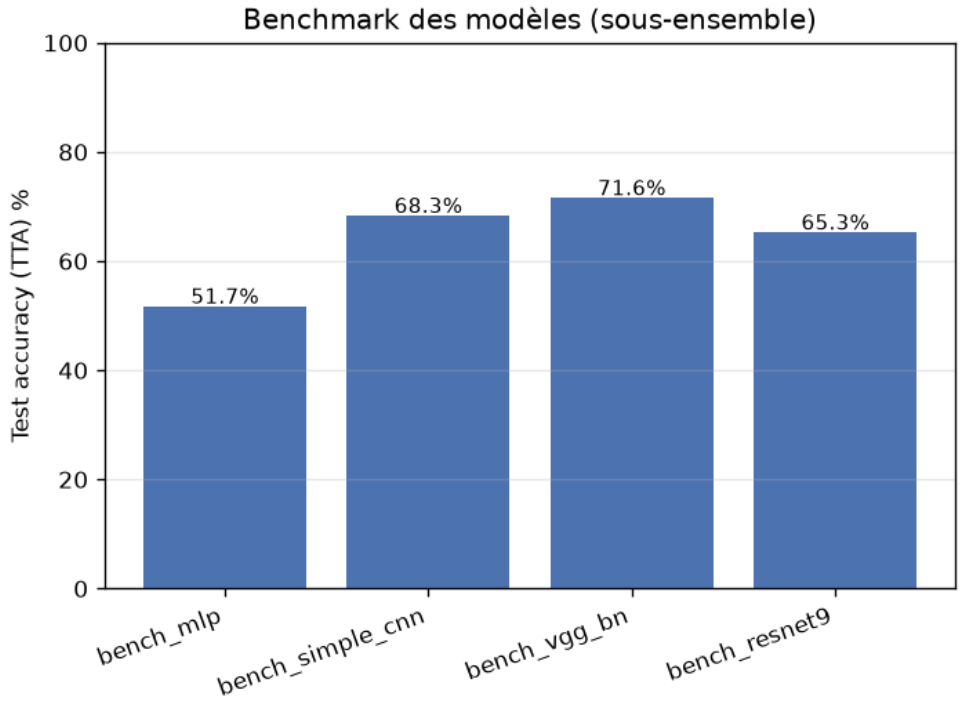


FIGURE 6 – Comparaison des modèles en phase de benchmark.

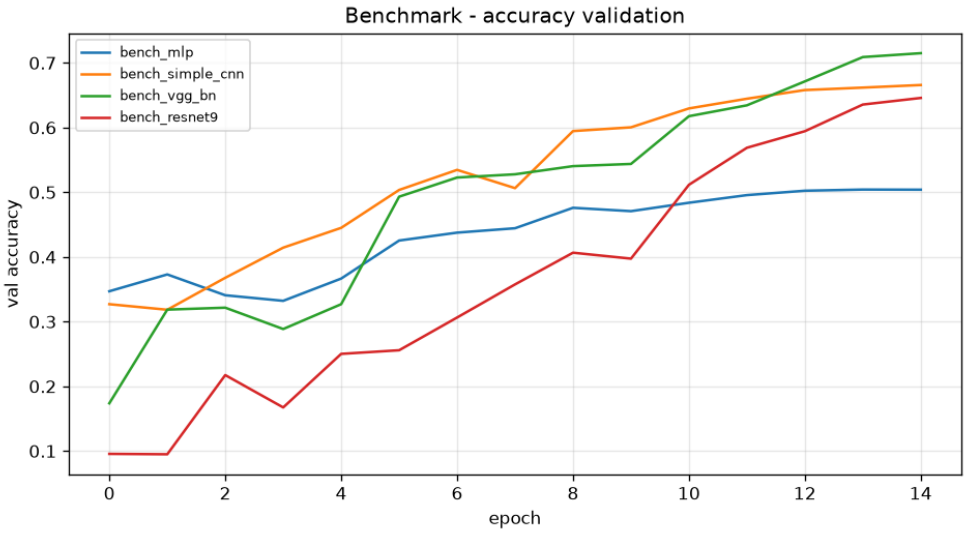


FIGURE 7 – Accuracy de validation par epoch des quatre modèles du benchmark.

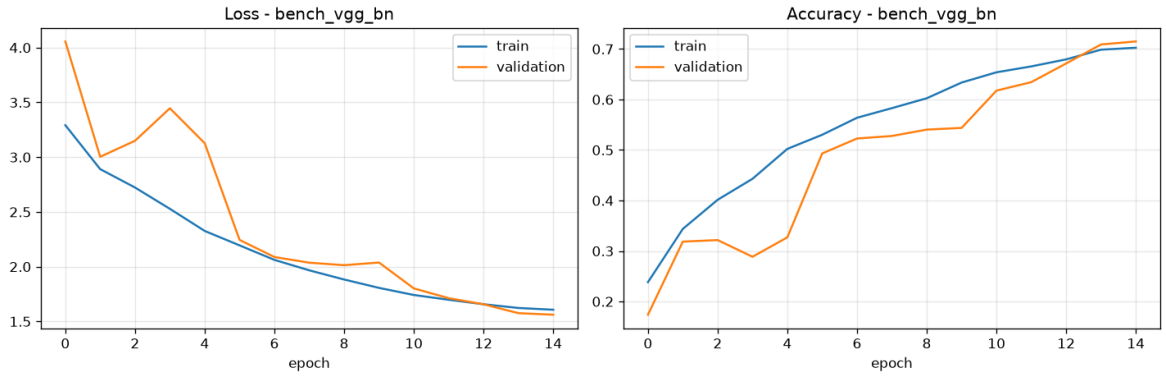


FIGURE 8 – Loss et accuracy par epoch – VGG-BN (benchmark).

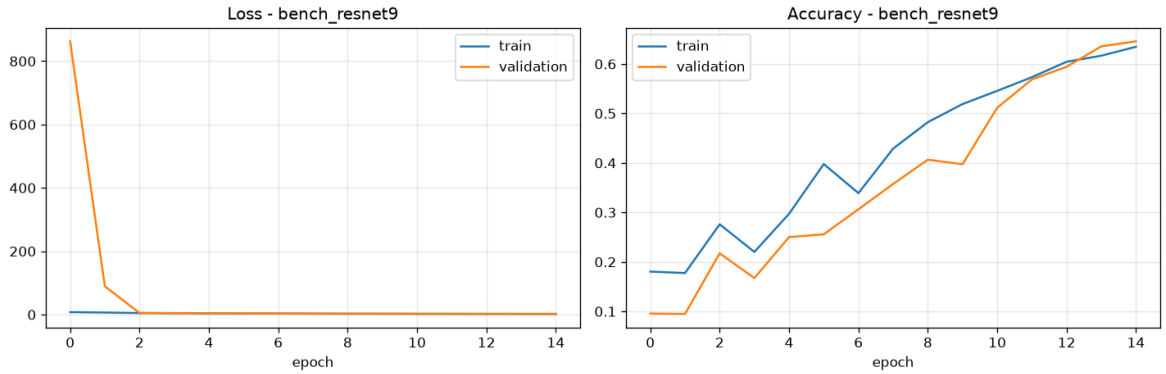


FIGURE 9 – Loss et accuracy par epoch – ResNet-9 (benchmark).

A.2 Phase 2 – Tuning du VGG-BN

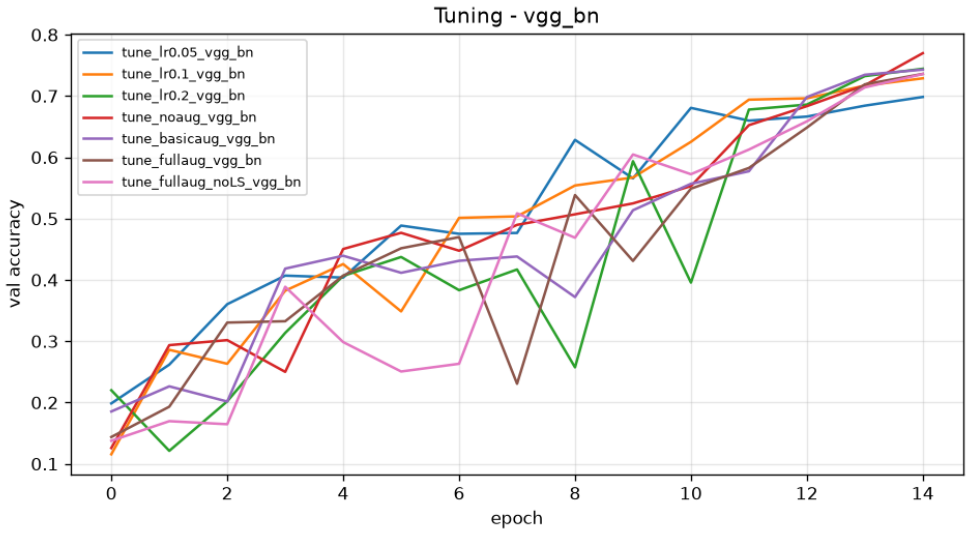


FIGURE 10 – Accuracy de validation par epoch : recherche du learning rate et ablation (VGG-BN).

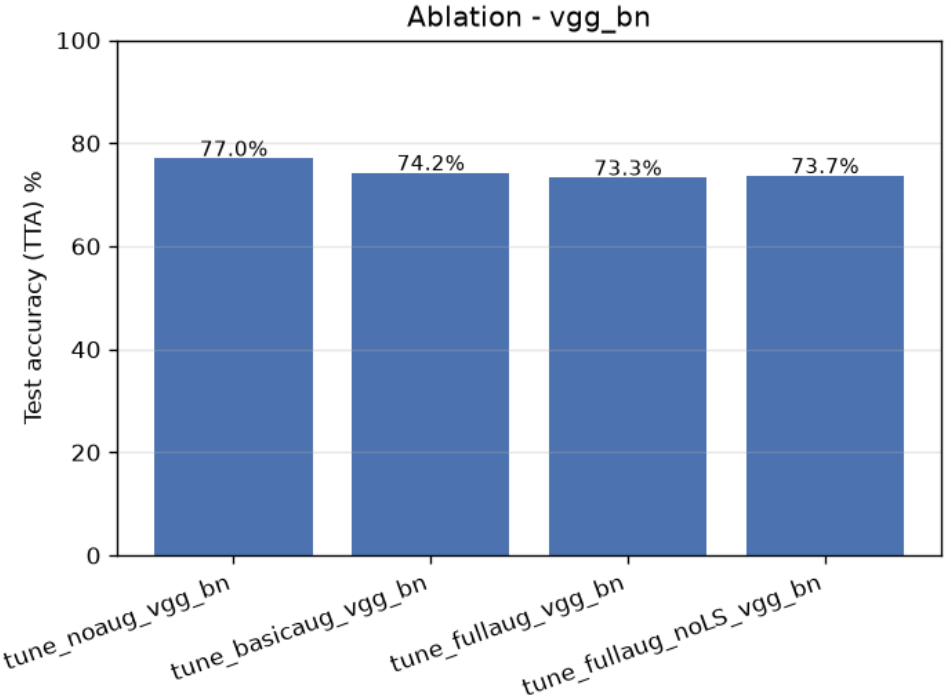


FIGURE 11 – Étude d'ablation : augmentation de données et label smoothing (VGG-BN).

A.3 Phase 3 – Modèles finaux (45 000 images, 40 epochs)

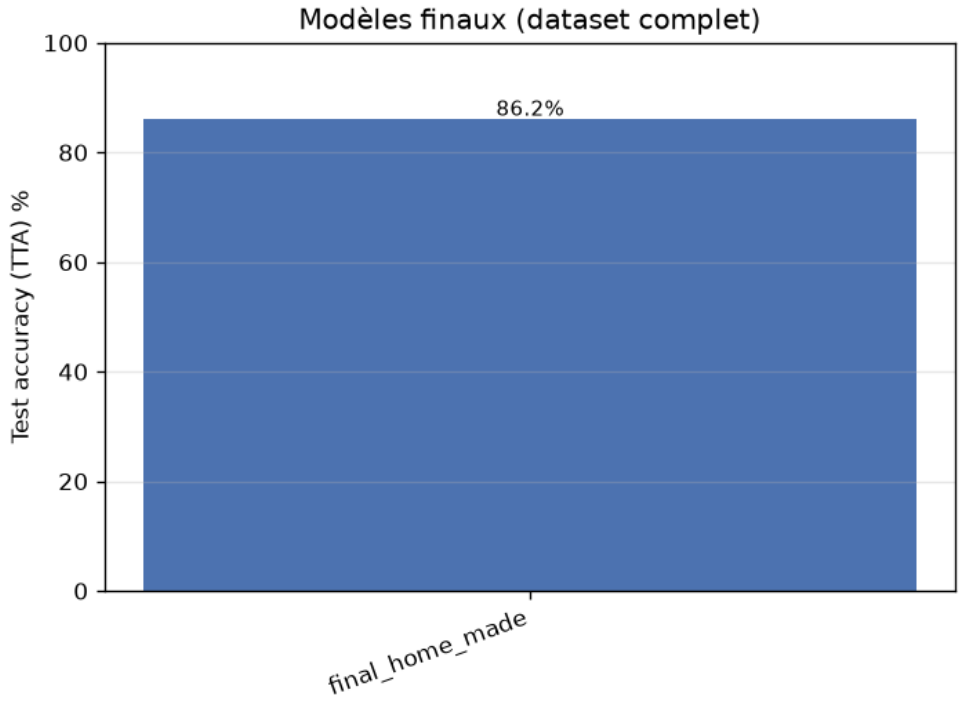


FIGURE 12 – Comparaison des modèles de la phase finale.

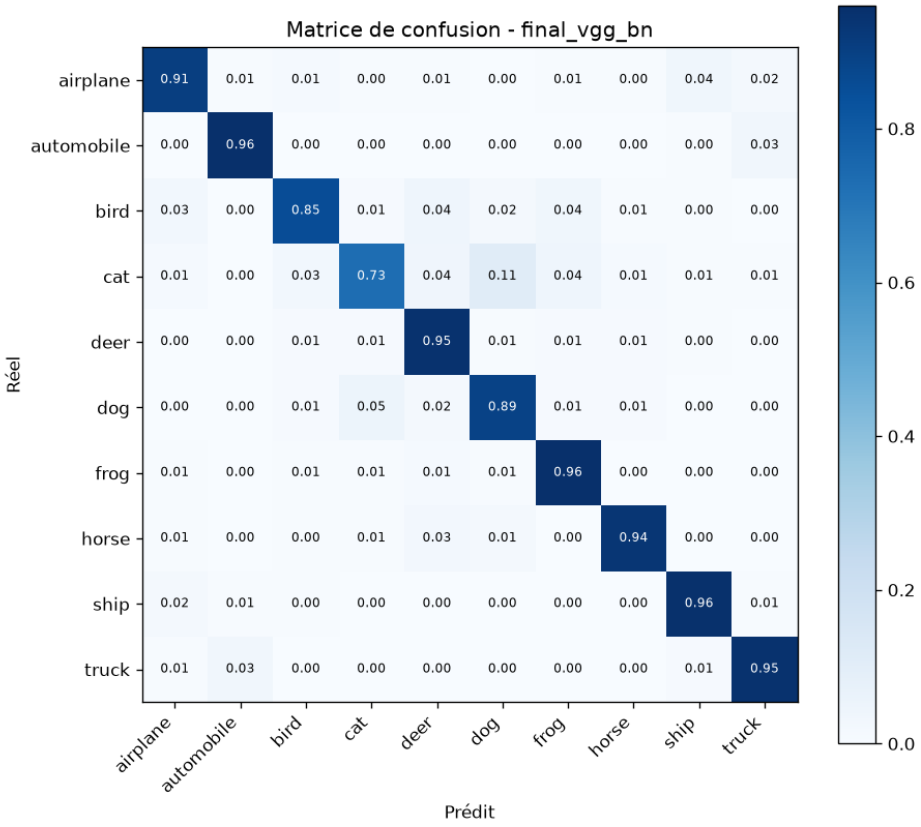


FIGURE 13 – Matrice de confusion normalisée – VGG-BN.

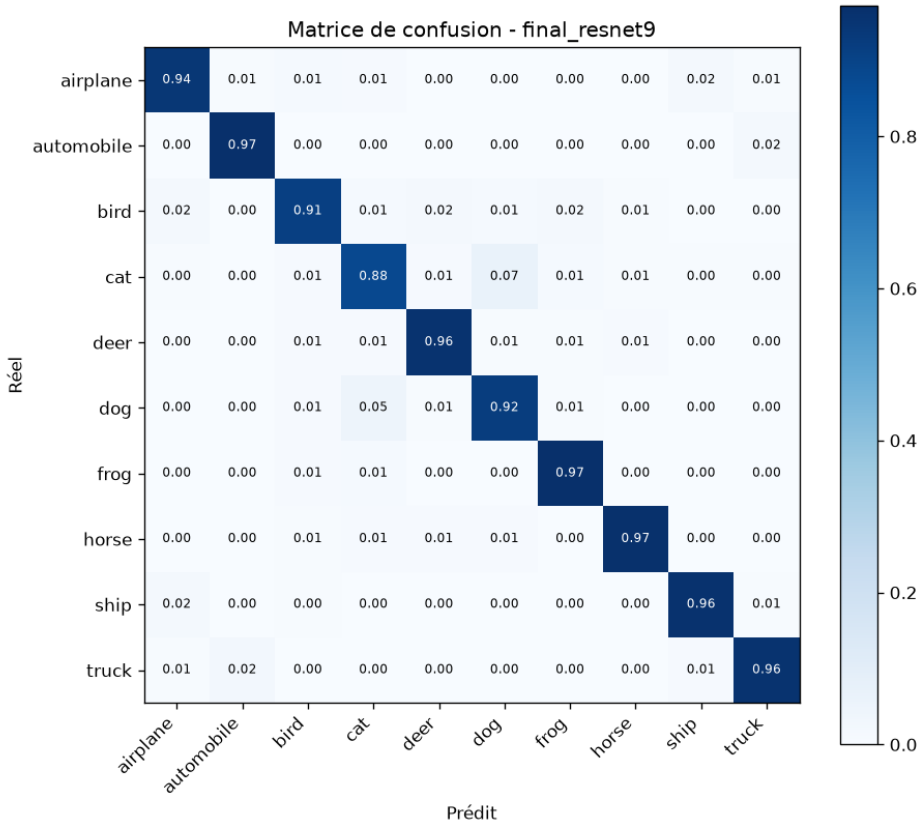


FIGURE 14 – Matrice de confusion normalisée – ResNet-9.

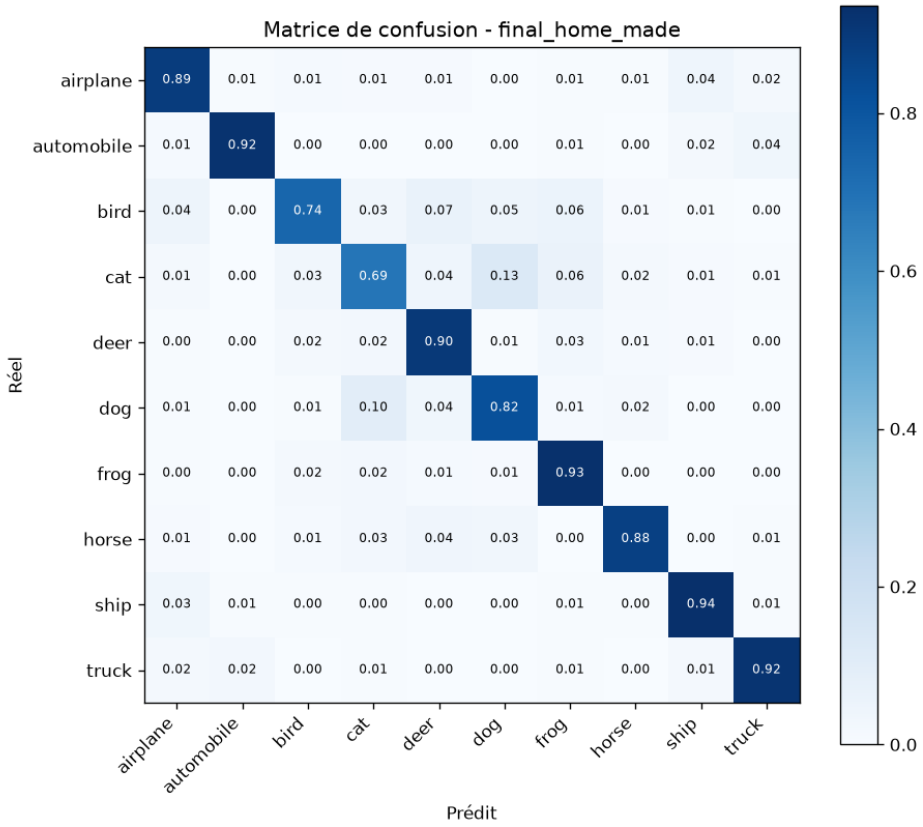


FIGURE 15 – Matrice de confusion normalisée – modèle personnalisé.

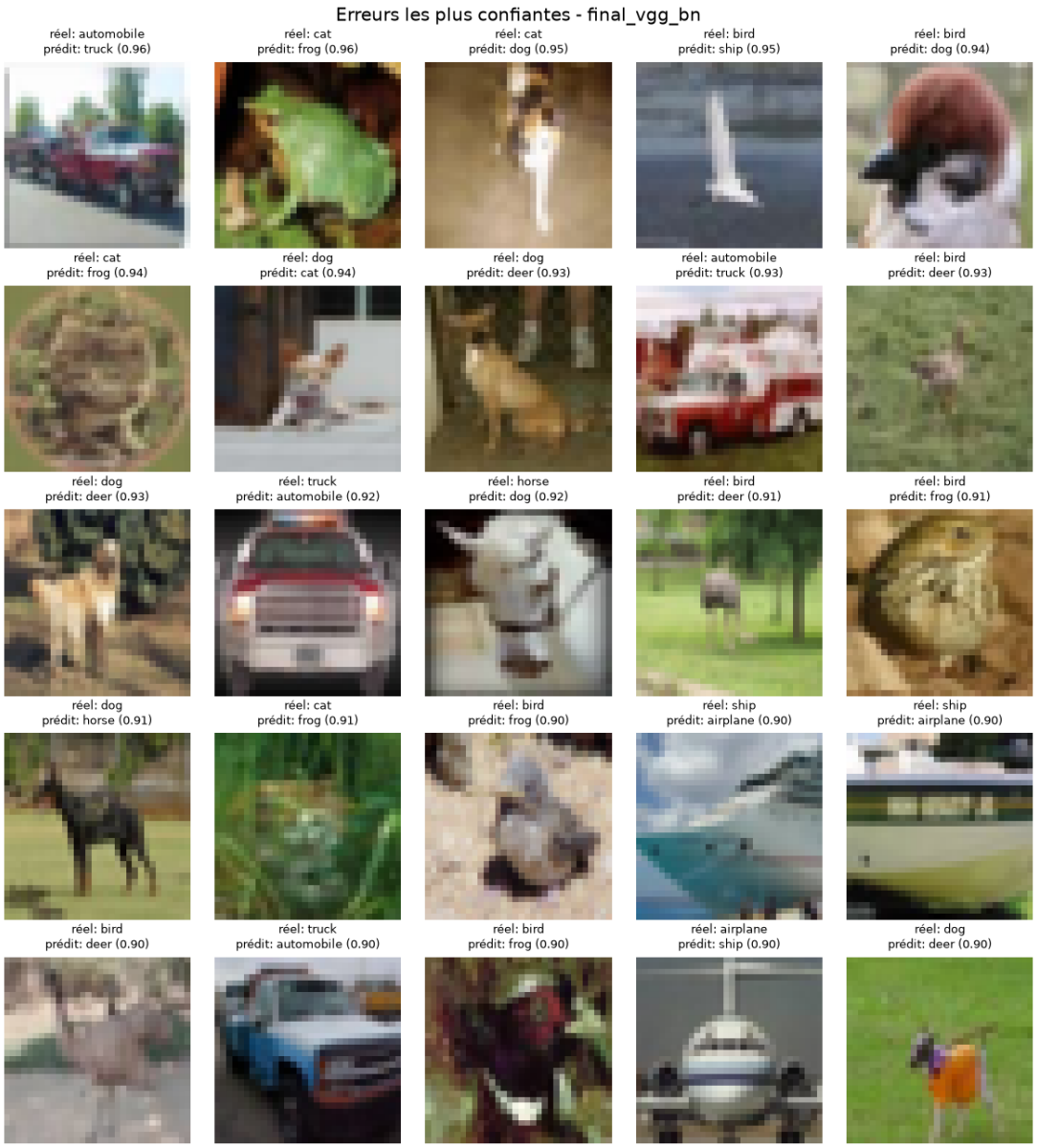


FIGURE 16 – Les 25 erreurs les plus confiantes – VGG-BN.

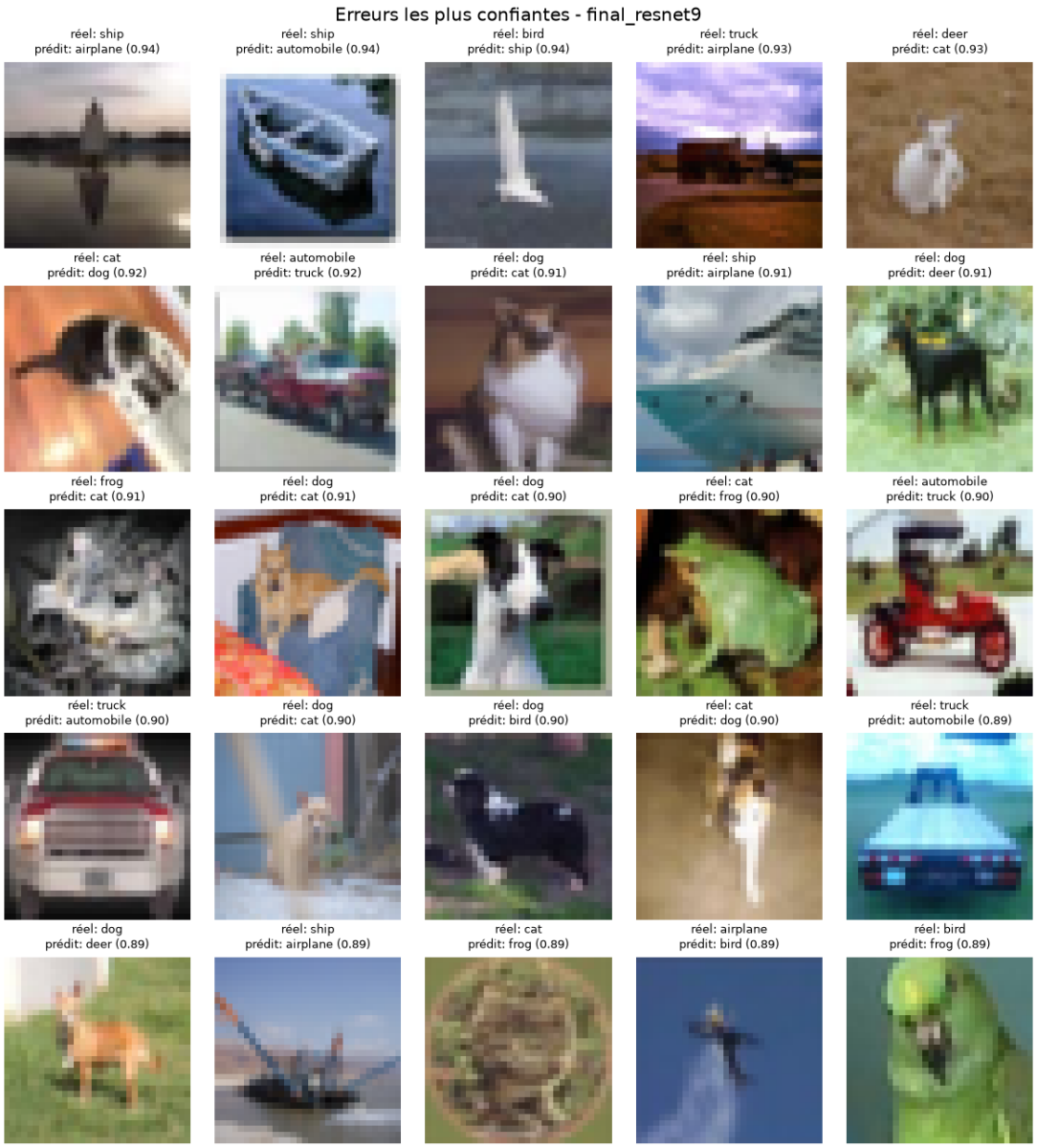


FIGURE 17 – Les 25 erreurs les plus confiantes – ResNet-9.

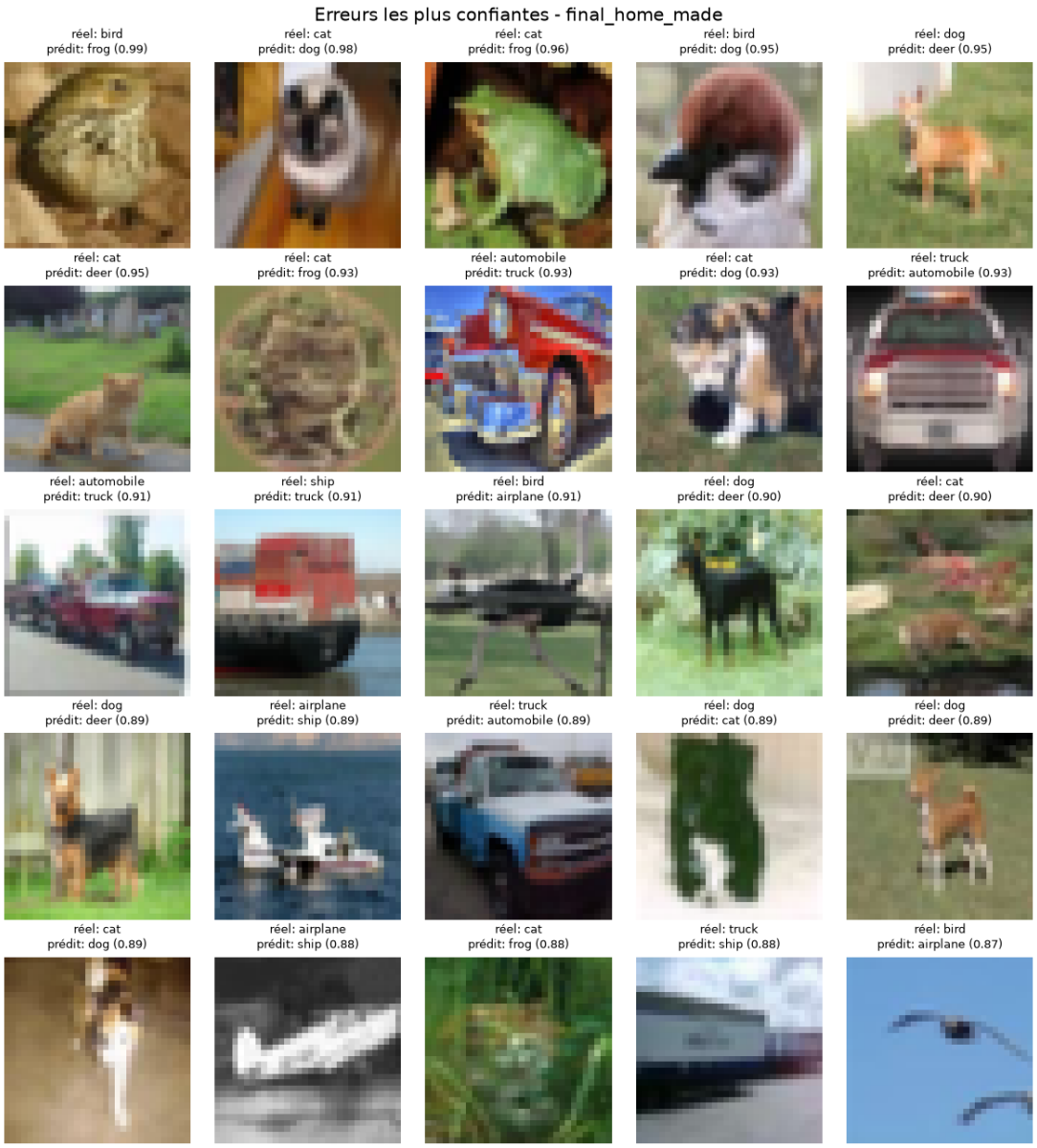


FIGURE 18 – Les 25 erreurs les plus confiantes – modèle personnalisé.

A.4 Tableau complet des résultats

Run	Images	Epochs	max_lr	Augm.	LS	Val. (%)	Test (%)	Test+TTA (%)	Temps (s)
bench_mlp	10 000	15	0,05	none	0,1	50,42	50,24	51,74	24
bench_simple_cnn	10 000	15	0,05	basic	0,1	66,58	68,12	68,34	58
bench_vgg_bn	10 000	15	0,10	full	0,1	71,50	71,43	71,58	386
bench_resnet9	10 000	15	0,10	full	0,1	64,58	65,19	65,27	1 140
tune_lr0.05_vgg_bn	10 000	15	0,05	full	0,1	69,84	69,26	69,27	420
tune_lr0.1_vgg_bn	10 000	15	0,10	full	0,1	72,88	71,59	71,61	329
tune_lr0.2_vgg_bn	10 000	15	0,20	full	0,1	74,44	73,84	74,26	336
tune_noaug_vgg_bn	10 000	15	0,20	none	0,1	76,98	76,40	77,00	338
tune_basicaug_vgg_bn	10 000	15	0,20	basic	0,1	74,30	74,05	74,24	337
tune_fullaug_vgg_bn	10 000	15	0,20	full	0,1	73,60	73,06	73,26	339
tune_fullaug_noLS_vgg_bn	10 000	15	0,20	full	0,0	73,58	73,10	73,66	333
final_vgg_bn	45 000	40	0,20	none	0,1	91,18	90,30	90,97	4 035
final_resnet9	45 000	40	0,10	full	0,1	93,92	94,03	94,45	9 104
final_home_made	45 000	40	0,05	basic	0,1	85,92	85,38	86,23	1 476

TABLE 11 – Ensemble des entraînements réalisés (d'après `results_summary.csv`). LS : label smoothing. La ligne `final_home_made` reprend les valeurs saisies manuellement.